

OPENJDK · PROJECT LOOM

OpenJDK y Loom

Un vistazo, actualizado a JDK 27

Octubre de 2026

¿Qué es OpenJDK?

La implementación de referencia, de código abierto, de la plataforma Java SE.

Mantenida por Oracle junto con la comunidad y otras empresas. Sobre ella se construyen la mayoría de las distribuciones de Java actuales.

2006

Sun anuncia la liberación de Java como software libre

2007

Se lanza oficialmente OpenJDK

INCLUYE

javac

Compilador

JVM (HotSpot)

Máquina virtual de Java

Bibliotecas estándar

java.base, java.net, java.util.concurrent...

Proyectos principales y su estado

Proyecto	Tipo de mejora	Estado a octubre de 2026
Amber	Lenguaje	Entrega continua; patrones primitivos en 5.ª preview (JDK 27)
Loom	Concurrencia	Hilos virtuales y Scoped Values finales; concurrencia estructurada en preview
Valhalla	Memoria y datos	Primera preview de value classes (JEP 401) en JDK 28
Lilliput	Memoria	Compact Object Headers activados por defecto en JDK 27
Panama	Integración con código nativo	API FFM final (JDK 22); Vector API aún en incubación
Leyden	Arranque y calentamiento	Caché AOT entregada en JDK 24, 25 y 26; falta código AOT
Skara openjdk.org	Infraestructura	Completado: el código vive en Git y GitHub desde 2020

Línea de tiempo · 2006–2018

2006–07

Nace OpenJDK. Sun libera Java como software libre (GPL) y en 2007 lanza OpenJDK.

2014

Java 8: lambdas y streams, giro hacia el estilo funcional. Arrancan **Valhalla** y **Panama**.

2017

Oracle anuncia un **ciclo de versiones cada seis meses**. Arrancan **Amber** y **Loom**.

2018

Java 10 (marzo), primera versión semestral; **Java 11** (septiembre), primera LTS del nuevo modelo.

Corrección respecto a la versión anterior: Valhalla y Panama comenzaron en 2014, no en 2017–2018.

Línea de tiempo · 2020–2023

2020

Skara: el código de OpenJDK migra a Git y GitHub. Se anuncia **Leyden**.

2021

Java 17 LTS: consolida records y pattern matching para instanceof (finales desde Java 16) y clases selladas. Arranca **Lilliput**.

2022

JDK 19: primera preview de los hilos virtuales de Loom.

2023

Java 21 LTS: hilos virtuales definitivos (JEP 444); pattern matching para switch y record patterns.

Línea de tiempo · 2024–2027

Mar 2024

JDK 22

API de funciones y memoria foránea (Panama) final.

Mar 2025

JDK 24

Caché AOT de Leyden (JEP 483). Hilos virtuales sin anclaje en synchronized (JEP 491).

Sep 2025 · LTS

Java 25

Scoped Values final. Compact Object Headers final. Ergonomía y perfiles AOT (JEP 514, 515).

Mar 2026

JDK 26

Caché AOT con cualquier GC, incluido ZGC (JEP 516). Concurrencia estructurada, 6.^a preview.

Sep 2026 · versión actual

JDK 27

Compact Object Headers y G1 por defecto. TLS 1.3 post-cuántico. Concurrencia estructurada, 7.^a preview.

Mar 2027 · próxima

JDK 28

Primera preview de las value classes de Valhalla (JEP 401).

LTS cada cuarta versión: 17 · 21 · 25 · la próxima es Java 29, prevista para septiembre de 2027.

PROJECT LOOM

Concurrencia masiva con código de estilo secuencial

openjdk.org/projects/loom

El problema y la respuesta de Loom

Hilos de plataforma

- Uno por cada hilo del sistema operativo
- Pila grande reservada y cambios de contexto costosos
- Un servidor «un hilo por petición» se queda en

^{miles}
≈ 1 000 hilos

Hilos virtuales

- Los gestiona la JVM, no el sistema operativo
- Se montan sobre pocos hilos portadores
- Al bloquearse en E/S se desmontan y liberan al portador

≈ 1 000 000 de tareas

Loom hoy: tres piezas que trabajan juntas

FINAL · JAVA 21

Hilos virtuales

Hilos baratos para tareas que pasan la mayor parte del tiempo esperando E/S.

Desde JDK 24 (JEP 491) ya no se anclan al portador dentro de **synchronized**.

7.^a PREVIEW · JDK 27

Concurrencia estructurada

Un grupo de subtareas se trata como una unidad de trabajo.

Si una falla, se cancelan las demás; ninguna sobrevive al bloque que las creó (JEP 533).

FINAL · JAVA 25

Scoped Values

Datos inmutables compartidos con los métodos llamados y los hilos hijos.

Vida acotada a un bloque; alternativa más barata y segura a ThreadLocal (JEP 506).

Idea común: la estructura del código refleja la estructura de la concurrencia.

Hilos virtuales en código

Un millón de tareas que duermen un segundo cada una. Con hilos de plataforma, esto agota la memoria.

```
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {  
    IntStream.range(0, 1_000_000).forEach(i ->  
        ex.submit(() -> {  
            Thread.sleep(Duration.ofSeconds(1));  
            return i;  
        }));  
} // close() espera a que terminen todas
```

Concurrencia estructurada y Scoped Values en código

StructuredTaskScope · preview

```
Response handle() throws InterruptedException {
    try (var scope = StructuredTaskScope.open())
    {
        var user  = scope.fork(() -> findUser());
        var order = scope.fork(() -> fetchOrder());
        scope.join(); // ambas, o falla
        return new Response(user.get(),
                             order.get());
    }
}
```

ScopedValue · final

```
static final ScopedValue<User> USER =
    ScopedValue.newInstance();

ScopedValue.where(USER, current)
    .run(() -> handle());

// en handle() y en sus subtareas:
User u = USER.get();
```

StructuredTaskScope sigue en preview: compilar y ejecutar con --enable-preview; la API puede cambiar.

Evolución de Loom

2017

Arranca el proyecto

2022

JDK 19: hilos virtuales en preview

2023

Java 21: hilos virtuales finales

2025

JDK 24: sin anclaje en synchronized

2025

Java 25: Scoped Values finales

2026

JDK 27: concurrencia estructurada, 7.^a preview

Lo pendiente: que la concurrencia estructurada salga de preview. Las fechas en azul marcan las piezas ya finales.

EN RESUMEN

OpenJDK es la implementación abierta de Java desde 2007.

Con Loom, Java pasa de miles a millones de tareas concurrentes sin abandonar el estilo secuencial.

Lenguajes con un modelo de evolución parecido

Evolucionan mediante propuestas formales, comunidad técnica y versiones periódicas. Son comparables a OpenJDK y sus JEP, aunque no iguales.

Python

PEP

Python Enhancement
Proposals

Rust

RFC

Requests for
Comments del
proyecto Rust

Go

Proposals

Propuestas de diseño
revisadas en abierto

C# / .NET

Diseño abierto

Evolución abierta del
lenguaje y del runtime

Hilos virtuales y sus equivalentes en otros lenguajes

¿Cómo maneja cada lenguaje miles o millones de tareas concurrentes sin un hilo del sistema operativo por tarea?

Lenguaje	Mecanismo	Modelo de ejecución	Disponible desde	Estilo de código
Java	Hilos virtuales (Project Loom)	Hilos de usuario M:N gestionados por la JVM, con pila propia	Java 21 (2023)	Secuencial y bloqueante, sin cambios
Go	Goroutines	Hilos de usuario M:N del runtime, con pila que crece dinámicamente	Go 1.0 (2012)	Secuencial; canales para comunicarse
C# / .NET	async / await sobre Task	Corrutinas sin pila (máquina de estados del compilador) sobre el ThreadPool	C# 5 (2012)	Requiere async/await en toda la cadena de llamadas
Python	asyncio (async / await)	Corrutinas sin pila en un bucle de eventos de un solo hilo	Python 3.5 (2015)	Requiere async/await y librerías compatibles
Rust	async / await + runtime externo (p. ej. Tokio)	Futures sin pila, evaluados por sondeo; el lenguaje no trae runtime	Rust 1.39 (2019)	Requiere async/await y elegir un runtime

Con pila (stackful): Java, Go

El código bloqueante de siempre escala; el runtime suspende y reanuda la tarea.

Sin pila (stackless): C#, Python, Rust

Las funciones asíncronas se marcan con async y se esperan con await ("funciones coloreadas").

Gracias

openjdk.org · openjdk.org/projects/loom